

！练习 5.3.2: 给出一个 SDD, 将一个带有 + 和 * 的中缀表达式翻译成没有冗余括号的表达式。比如, 因为两个运算符都是左结合的, 并且 * 的优先级高于 +, 所以 $((a * (b + c)) * (d))$ 可翻译为 $a * (b + c) * d$ 。

产生式	语义规则
1) $L \rightarrow E_n$	$L.expr = E.expr$
2) $E \rightarrow E' + T$	$E.prec = 1$ $if\ E'.prec < 1 : S_1 = (E'.expr)$ $else : S_1 = E'.expr$ $if\ T.prec \leq 1 : S_2 = (T.expr)$ $else : S_2 = T.expr$ $E.expr = S_1 + S_2$
3) $E \rightarrow T$	$E.prec = T.prec$ $E.expr = T.expr$
4) $T \rightarrow T' * F$	$T.prec = 2$ $if\ T'.prec < 2 : S_1 = (T'.expr)$ $else : S_1 = T'.expr$ $if\ F.prec \leq 2 : S_2 = (F.expr)$ $else : S_2 = F.expr$ $T.expr = S_1 * S_2$
5) $T \rightarrow F$	$T.prec = F.prec$ $T.expr = F.expr$
6) $F \rightarrow (E)$	$F.prec = E.prec$ $F.expr = E.expr$
7) $F \rightarrow id$	$F.prec = 3$ $F.expr = id.val$

！练习 5.4.3: 下面的 SDT 计算了一个由 0 和 1 组成的串的值。它把输入的符号串当作按照正二进制数来解释。

$$B \rightarrow \begin{matrix} B_1\ 0 \{B.val = 2 \times B_1.val\} \\ B_1\ 1 \{B.val = 2 \times B_1.val + 1\} \\ 1 \{B.val = 1\} \end{matrix}$$

改写这个 SDT, 使得基础文法不再是左递归的, 但仍然可以计算出整个输入串的相同的 $B.val$ 的值。

S 属性 SDT $\Leftrightarrow$ LR 分析

合并产生式:

L 属性 SDT $\Rightarrow$ LL 分析

$$B \rightarrow B_1\ bit \{ B.val = 2 \times B_1.val + bit.val \}$$

$$| \quad 1 \{ B.val = 1 \}$$

$$bit \rightarrow 0 \{ bit.val = 0 \}$$

$$| \quad 1 \{ bit.val = 1 \}$$

消除左递归:

i: 继承属性

$$B \rightarrow 1 \{ A.i = 1 \} A \quad val: 综合属性$$

$$A \rightarrow bit \{ A_1.i = 2 \times A.i + bit.val \} A_1 \{ A.val = A_1.val \}$$

$$| \quad \epsilon \{ A.val = A.i \}$$

$$bit \rightarrow 0 \{ bit.val = 0 \}$$

$$| \quad 1 \{ bit.val = 1 \}$$